UNIVERSIDAD EAFIT -
ESCUELA DE CIENCIAS APLICADAS E INGENIERÍA

DEPARTAMENTO DE INFORMÁTICA Y ANALÍTICA

ASIGNATURA: SISTEMAS OPERATIVOS - PERIODO 2026-2

EVALUACIÓN PARCIAL PRÁCTICA - TEÓRICA (VALOR: 25%)

Tema: Seguridad de Archivos mediante Intervención en la Capa del Kernel y Mapeo de Memoria (mmap)
Referencia Base: Repositorio GitHub SO2026B - CapaKernel
Modalidad: Individual o Equipo
Tiempo estimado: 2 semanas

1. CONTEXTO Y DESCRIPCIÓN DEL PROBLEMA

En los sistemas operativos modernos de la familia UNIX/Linux, el acceso a archivos mediante las llamadas al sistema convencionales (`read`, `write`, `pread`, `pwrite`) introduce sobrecostos de cambios de contexto entre espacio de usuario (Ring 3) y espacio de kernel (Ring 0), además del copiado redundante de buffers entre la memoria caché de páginas del kernel (Page Cache) y la memoria del proceso.

La llamada al sistema mmap() permite mapear archivos o dispositivos directamente en el espacio de direcciones virtuales de un proceso, posibilitando accesos a través de punteros y aprovechando el mecanismo de paginación por demanda (Page Faults) gestionado por la MMU. Sin embargo, este mecanismo plantea desafíos críticos de seguridad e integridad:

- Exposición de páginas compartidas en memoria física (`MAP_SHARED` vs. `MAP_PRIVATE`).
- Posibilidad de ataques tipo race condition (e.g., Dirty COW / CVE-2016-5195) aprovechando la interacción entre Copy-On-Write (COW) y la desprotección de páginas.
- Falta de mecanismos transparentes de cifrado e inspección a nivel de bloque/página cuando los procesos acceden directamente a las regiones mapeadas sin pasar por los puntos tradicionales de intercepción de I/O de usuario.

El reto: Diseñar e implementar una solución en la **capa de Kernel** que refuerce la seguridad, integridad y control de acceso sobre archivos mapeados con mmap(), garantizando aislamiento de memoria, detección/prevención de modificaciones no autorizadas y auditoría de accesos.

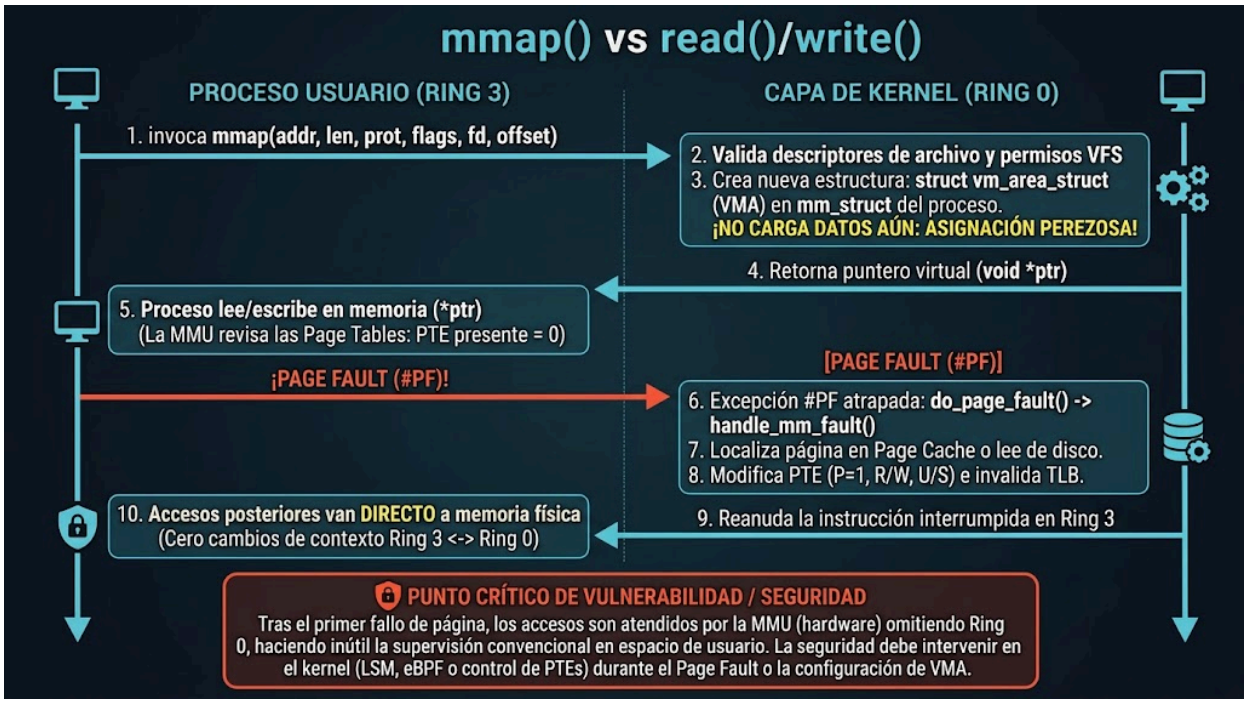
1.1 INFOGRAMA CONCEPTUAL: ARQUITECTURA DE ANILLOS DE PRIVILEGIO (RINGS) Y FRONTERA DE SEGURIDAD

Tabla Comparativa de Dominios de Ejecución y Privilegios

Anillo / Nivel	Modo de CPU	Permisos de Instrucciones	Acceso a Memoria	Comportamiento ante Violación
Ring 3 (Usuario)	CPL = 3 (No privilegiado)	Solo instrucciones de propósito general. Prohibidas: `CLI`, `STI`, `LIDT`, `MOV CRx`.	Aislado a su propio espacio virtual (`0x0000000000000000` a `0x00007FFFFFFFFFFF` en x86_64).	Excepción de Protección General (`#GP`) o Fallo de Segmentación (`SIGSEGV`).
Ring 1 & 2	CPL = 1, 2	Acceso intermedio; rara vez implementado por costos de rendimiento en transiciones de anillo.	No aplicable en kernels contemporáneos (modelo de memoria plana).	N/A en Linux.
Ring 0 (Kernel)	CPL = 0 (Supervisor)	Total: manipulación de registros de control, interrupciones, TLB, páginas maestras.	Espacio virtual total, acceso directo al mapa físico (`phys_to_virt()`, memoria kernel).	o compromiso total del sistema si falla.

1.2 INFOGRAMA TÉCNICO: ANATOMÍA DE UN ACCESO MMAP() FRENTE A READ()/WRITE()

Esquema de Flujo Operativo y Ruptura de la Frontera de Seguridad



Síntesis del Desafío para el Parcial

Como lo ilustra el esquema de la **Fase 3**, una vez que la entrada en la tabla de páginas (PTE) está activa, la lectura y escritura sobre el archivo proyectado se ejecuta **a velocidad de hardware sin conmutación de contexto a Ring 0**.
Por este motivo, para auditar, cifrar, limitar o detectar anomalías sobre los datos:

- **La defensa no puede residir en espacio de usuario**, pues cualquier biblioteca envoltorio (LD_PRELOAD) es trivialmente eludible por un atacante.
- **La solución debe arraigarse en la capa de Kernel**, interceptando el alta del VMA (`mmap`/LSM hooks), instrumentando el controlador de fallos de página (`handle_mm_fault`), o monitoreando eventos de dirtying y sincronización (`msync`/eBPF).

2. PARTE I: ANÁLISIS CONCEPTUAL Y PREGUNTAS DIAGNÓSTICAS (10 PREGUNTAS - 30%)

El estudiante/equipo debe responder y fundamentar con rigor técnico y a nivel de arquitectura del SO las siguientes diez preguntas orientadas a desglosar y resolver el problema planteado:

1. **Mapeo Virtual y Page Faults:** ¿Cuál es la secuencia exacta de eventos a nivel de kernel y MMU desde que un proceso invoca `mmap()` hasta que ocurre el primer fallo de página (Page Fault) por lectura o escritura en una página mapeada?
2. **MAP_SHARED vs. MAP_PRIVATE:** ¿Cómo gestiona el subsistema de memoria virtual de Linux (`struct vm_area_struct`) la diferencia entre banderas `MAP_SHARED` y `MAP_PRIVATE`, y cuáles son las implicaciones de seguridad respecto a la persistencia y visibilidad cruzada entre procesos?
3. **Mecanismo Copy-On-Write (COW):** Describa el ciclo de vida de una página bajo COW. ¿En qué condiciones una falla en la gestión de banderas de protección (`VM_MAYWRITE`, `PROT_READ`, `PROT_WRITE`) en el kernel puede derivar en una escalada de privilegios o corrupción de archivos del sistema?
4. **Intercepción de Llamadas en el Kernel:** Para monitorear o restringir el uso de `mmap()` sobre archivos sensibles, ¿cuáles son las ventajas, desventajas y viabilidad de utilizar un módulo de kernel (LKM) con hook en la System Call Table frente a un Linux Security Module (LSM) con ganchos (hooks) como `file_mmap`?
5. **Cifrado Transparente en Memoria:** Si un archivo confidencial se encuentra cifrado en disco, ¿cómo debe interactuar la capa de Page Cache y el driver de sistema de archivos en el kernel para que la proyección en `mmap()` sea legible en el espacio de usuario autorizado sin exponer el contenido descifrado a otros procesos no privilegiados en memoria física?
6. **Sincronización e Integridad (`msync` y `fsync`):** ¿Cómo garantiza el kernel la consistencia de páginas marcadas como sucias (*dirty pages*) en memoria antes de ser persistidas en disco, y cómo puede un atacante evadir filtros de seguridad si los datos no son inspeccionados en el momento de la escritura en memoria?
7. **Control de Accesos con eBPF:** ¿Cómo se puede emplear eBPF (Extended Berkeley Packet Filter) y `kprobes`/`fexit` para auditar accesos indebidos o patrones anómalos de lectura/escritura sobre regiones mapeadas con `mmap()` sin recompilar el núcleo del sistema operativo?
8. **Evasión de Permisos POSIX:** Si un proceso abre un descriptor de archivo con permisos de solo lectura (`O_RDONLY`), ¿puede bajo ciertas configuraciones de kernel mapearlo con `PROT_READ | PROT_WRITE` y `MAP_PRIVATE`? Explique por qué el kernel lo permite y qué riesgos de fuga de información o alteración temporal de datos implica.
9. **Mitigación de Ataques de Carrera (Race Conditions):** ¿Cómo previene el kernel actual condiciones de carrera concurrentes entre `madvise(MADV_DONTNEED)` y escrituras sobre páginas mapeadas privadas?
10. **Aislamiento de Memoria y Paginación:** ¿Qué papel juegan las estructuras de descriptores de página (`struct page`), tablas de páginas (PGD, P4D, PUD, PMD, PTE) y los bits de protección de la MMU (NX, R/W, U/S) en la contención de inyecciones de código malicioso en memoria mapeada?

3. PARTE II: DISEÑO DE LA ESTRATEGIA CON APOYO DE INTELIGENCIA ARTIFICIAL (30%)

En esta sección, el estudiante debe formular una estrategia formal para resolver el problema de seguridad, incorporando explícitamente herramientas de Inteligencia Artificial (LLMs, asistentes de generación de código, herramientas de análisis estático asistidas por IA) como acelerador del ciclo de ingeniería de software de sistemas.

Debe documentar y evidenciar los siguientes componentes:

3.1. Definición del Flujo de Trabajo Asistido por IA

- Rol de la IA en la fase de exploración arquitectónica (análisis de estructuras de kernel, cabeceras en C, diseño de hooks).
- Rol de la IA en la generación y verificación de código en C de bajo nivel (módulos LKM, programas eBPF o capas de intercepción).
- Rol de la IA en la creación de casos de prueba y exploits controlados (PoC - Proof of Concept) para validar vulnerabilidades en `mmap()`.

3.2. Metodología de Ingeniería de Prompts y Verificación Crítica

- Tabla con al menos 4 prompts estructurados utilizados durante la formulación de la solución (indicando contexto, rol, restricción y resultado esperado).
- Análisis de alucinaciones técnicas detectadas: casos donde el modelo sugirió funciones obsoletas, no exportadas del kernel (`EXPORT_SYMBOL`), macros inexistentes o prácticas inseguras (e.g., deshabilitar el bit de protección contra escritura de CR0 sin control de concurrencia).
- Procedimiento de validación manual y compilación contra los headers del kernel de la distribución utilizada.

3.3. Estrategia Arquitectónica Seleccionada

- Diagrama conceptual y descripción de la arquitectura propuesta (módulo de kernel, gancho LSM / eBPF, o wrapper en espacio de kernel).
- Matriz de compensaciones (*trade-offs*): Rendimiento (latencia de I/O y Page Faults) vs. Seguridad (profundidad de inspección y cobertura).

4. PARTE III: IMPLEMENTACIÓN, PRESENTACIÓN Y ARGUMENTACIÓN DE LA SOLUCIÓN (40%)

El estudiante/equipo debe presentar la solución técnica implementada y defenderla técnicamente:

4.1. Entregables de Código e Implementación

- Código fuente en lenguaje C debidamente modularizado y comentado, compatible con Linux Kernel 5.x / 6.x.
- Demostración de intercepción, validación de permisos o cifrado/descifrado seguro de regiones `mmap()`.
- Makefile de compilación que genere el módulo o binarios correspondientes sin advertencias (*clean build*).
- Scripts de prueba (Bash / Python) que simulen intentos de acceso no autorizado, ataques de carrera o modificaciones indebidas, demostrando la efectividad de la defensa implementada.

4.2. Argumentación y Análisis de Resultados

- Justificación teórica: ¿Por qué la solución en la capa de kernel seleccionada es óptima frente a soluciones en espacio de usuario?
- Medición de impacto: Evaluación del sobrecosto (*overhead*) introducido en las llamadas `mmap()` y en la tasa de atención de Page Faults con herramientas como `perf`, `strace`, o `ftrace`.
- Limitaciones y trabajo futuro: Análisis de vectores de ataque que aún queden descubiertos y propuestas de mitigación a nivel de hardware o hipervisor.

5. RÚBRICA DE EVALUACIÓN (25% DE LA NOTA DEFINITIVA)

Criterio		Nivel Excelente (4.5 - 5.0)	Nivel Aceptable (3.0 - 4.4)	Nivel Insuficiente (0.0 - 2.9)
Respuestas a las 10 Preguntas Diagnósticas	30%	Responde con profundidad rigurosa, citando estructuras internas del kernel (`vm_area_struct`, MMU, PTEs) y flujo exacto de eventos.	Responde a la mayoría de preguntas correctamente, con explicaciones conceptuales pero superficiales en detalles de kernel.	Respuestas erróneas, incompletas o copiadas literalmente sin dominio del funcionamiento de memoria virtual.
Estrategia y Uso Metodológico de IA	30%	Evidencia un uso crítico de IA, documenta prompts, identifica alucinaciones técnicas de bajo nivel y define una arquitectura sólida.	Utiliza IA pero de forma meramente instrumental, con poca documentación de prompts o escasa validación de alucinaciones.	No documenta el uso de IA o presenta código generado por IA sin comprender su funcionamiento ni

Criterio		Nivel Excelente (4.5 - 5.0)	Nivel Aceptable (3.0 - 4.4)	Nivel Insuficiente (0.0 - 2.9)
				validar errores de compilación.
Solución Técnica en Kernel y Argumentación	40%	Código funcional en C, compila en kernel moderno, bloquea o asegura `mmap()` con éxito, e incluye análisis exhaustivo de métricas y defensa técnica.	Código funcional con bugs menores o pruebas parciales; argumentación básica con mediciones incompletas de rendimiento.	Código no compila o presenta kernel panics; no implementa mecanismos reales de kernel ni sustenta adecuadamente la solución.

Instrucciones de Entrega:

- Subir el repositorio de GitHub o paquete comprimido con el código fuente, Makefile y Readme explicativo.
- Enlace al presente documento o PDF con el informe completo de respuestas, estrategia con IA y argumentación.
- Cada miembro del equipo debe publicar la evidencia en el buzón de interactiva EAFIT